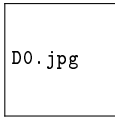


Lecture 36: Quiz 2 Revision

Solutions + Explanations (SQL)

Z2004: Database Management Systems



D0.jpg

IIT Madras Zanzibar

Thursday, 18 June 2026

How we'll revise Quiz 2

- ▶ We'll cover **what** each question is really testing.
- ▶ Then a **model answer**.
- ▶ Then **why the tempting wrong answers / wrong approaches fail**.

Main topics: aggregation, joins, subqueries, CTEs, window functions, stored procedures, transactions.

Section A (MCQs): Quick answer map

From the official answer key:

Q	Ans	Core idea
1	(b)	HAVING filters <i>groups</i> (after aggregation).
2	(b)	LEFT JOIN keeps all left rows; NULLs on missing matches.
3	(b)	COUNT(*) counts rows even when Grade is NULL.
4	(c)	DENSE_RANK ties w/out gaps; RANK leaves gaps.
5	(b)	CTE syntax begins with WITH.
6	(c)	DISTINCT removes duplicates in output rows.
7	(b)	IN subquery returns students who appear in Enrols.
8	(b)	SQL order: FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY.
9	(b)	Stored procedure can have control flow; view is a SELECT.
10	(b)	ROLLBACK undoes changes in a transaction.

MCQ deep-dive: the 3 most common traps

Trap 1: WHERE vs HAVING

- ▶ WHERE filters **rows** before grouping.
- ▶ HAVING filters **groups** after GROUP BY.

Trap 2: LEFT JOIN + WHERE

- ▶ If you do `LEFT JOIN ... WHERE right.col = ...` you often destroy the “leftness”.
- ▶ Put the filter in the ON clause when you need to keep unmatched left rows.

Trap 3: Ranking functions

- ▶ ROW_NUMBER: never ties.
- ▶ RANK: ties + gaps.
- ▶ DENSE_RANK: ties + no gaps.

Section B Q11: WHERE vs HAVING

What to write: definition + example for each.

Model points:

- ▶ WHERE: filters rows before grouping; cannot use aggregates.
- ▶ HAVING: filters groups after grouping; used with aggregates.

Example snippets:

```
-- WHERE before GROUP BY
SELECT ProgramCode, COUNT(*)
FROM Student
WHERE YearOfStudy = 2
GROUP BY ProgramCode;
```

```
-- HAVING after GROUP BY
SELECT CourseCode, COUNT(*)
FROM Enrols
GROUP BY CourseCode
HAVING COUNT(*) >= 2;
```

Q11: what loses marks in exams

- ▶ Writing aggregate conditions in WHERE (invalid SQL).
- ▶ Forgetting that WHERE changes the rows *before* counts/averages are computed.
- ▶ Mixing up “row filter” vs “group filter” in explanations.

Section B Q12: Window function explanation

Focus: interpreting `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`.

Key explanations to include:

- ▶ `PARTITION BY StudentID`: numbering restarts per student.
- ▶ `ORDER BY Grade ASC`: defines which row is 1st/2nd/3rd inside each student.
- ▶ `ROW_NUMBER` never ties; use `RANK/DENSE_RANK` for ties.

Section B Q13: Stored procedure usp_EnrolStudent

Goal: insert into Enrols with parameters; grade starts NULL.

```
CREATE PROCEDURE usp_EnrolStudent
    @StudentID INT,
    @CourseCode NVARCHAR(10),
    @Semester NVARCHAR(10)
AS
BEGIN
    INSERT INTO Enrols(StudentID, CourseCode, Semester, Grade)
    VALUES (@StudentID, @CourseCode, @Semester, NULL);
END;
```

Pitfalls:

- ▶ Wrong column order.
- ▶ Forgetting semester parameter.
- ▶ Accidentally inserting a grade instead of NULL.

Section B Q14: Transaction + rollback (concept)

What the examiner wants: show you understand atomicity.

```
BEGIN TRANSACTION;  
  
-- ... some INSERT/UPDATE/DELETE statements ...  
  
-- If something goes wrong:  
ROLLBACK;  
  
-- If everything is correct:  
-- COMMIT;
```

Common mistakes:

- ▶ Confusing ROLLBACK with DELETE.
- ▶ Forgetting that changes are not permanent until COMMIT.

Section C Q16: Dept course + enrolment counts (model)

Target output: include departments with zero courses and courses with zero enrolments.

```
SELECT d.DeptCode, d.DeptName,
       COUNT(DISTINCT c.CourseCode) AS NumCourses,
       COUNT(e.StudentID) AS NumEnrolmentsEven2026
FROM Department d
LEFT JOIN Course c
  ON c.DeptCode = d.DeptCode
LEFT JOIN Enrols e
  ON e.CourseCode = c.CourseCode
 AND e.Semester = 'Even2026'
GROUP BY d.DeptCode, d.DeptName;
```

Q16: key reasoning + mistakes

- ▶ Must start from Department and use **LEFT JOIN** to keep zero rows.
- ▶ Semester filter belongs in ON for the Enrols join.
- ▶ NumCourses should count distinct courses.

Section C Q17: 3+ graded courses in Even2026

```
SELECT s.StudentID, s.Name,  
       COUNT(DISTINCT e.CourseCode) AS NumCourses  
FROM Student s  
JOIN Enrols e  
  ON e.StudentID = s.StudentID  
WHERE e.Semester = 'Even2026'  
      AND e.Grade IS NOT NULL  
GROUP BY s.StudentID, s.Name  
HAVING COUNT(DISTINCT e.CourseCode) >= 3  
ORDER BY NumCourses DESC, s.StudentID;
```

Section C Q18: Top 5 courses by distinct students (include zeros)

```
SELECT TOP 5 c.CourseCode, c.Title,
             COUNT(DISTINCT e.StudentID) AS NumStudents
FROM Course c
LEFT JOIN Enrols e
  ON e.CourseCode = c.CourseCode
 AND e.Semester = 'Even2026'
GROUP BY c.CourseCode, c.Title
ORDER BY NumStudents DESC, c.CourseCode;
```

Section C Q19: Total enrolments across all semesters (include zeros)

```
SELECT s.StudentID, s.Name,
       COUNT(e.CourseCode) AS TotalEnrolments
FROM Student s
LEFT JOIN Enrols e
  ON e.StudentID = s.StudentID
GROUP BY s.StudentID, s.Name
ORDER BY TotalEnrolments DESC, s.StudentID;
```

Section C Q20: Courses with ≥ 2 different programmes (Even2026)

```
SELECT c.CourseCode, c.Title,
       COUNT(DISTINCT s.ProgramCode) AS NumProgrammes
FROM   Course c
JOIN   Enrols e
      ON e.CourseCode = c.CourseCode
JOIN   Student s
      ON s.StudentID = e.StudentID
WHERE  e.Semester = 'Even2026'
GROUP BY c.CourseCode, c.Title
HAVING COUNT(DISTINCT s.ProgramCode) >= 2
ORDER BY NumProgrammes DESC, c.CourseCode;
```

Wrap-up: patterns to memorize

- ▶ **LEFT JOIN + filter-in-ON** to keep zeros.
- ▶ **GROUP BY + HAVING** for “at least N”.
- ▶ **COUNT(DISTINCT ...)** when asked for distinct students/courses/programmes.
- ▶ **Ranking/window functions**: pick the right tool for ties.